Deep Learning Image Classification Report

Caltech-101 Dataset: ResNet50, EfficientNetB0, and Vision Transformer Comparison

Date: March 1, 2026
Author: Jingxi Zhang
Project: Image Classification Using Deep Learning Methods

Github Link: https://github.com/JingxiZhang-77/SHBT261-miniProject1

Executive Summary

This report presents a comprehensive evaluation of three state-of-the-art deep learning architectures for image classification on the Caltech-101 dataset. The models evaluated are:

- **ResNet50**: Residual Networks with 50 layers
- **EfficientNetB0**: Mobile-Efficient Networks
- **Vision Transformer (ViT-Base)**: Transformer-based architecture

All models leverage transfer learning with ImageNet pre-trained weights and are evaluated using comprehensive metrics including accuracy, per-class accuracy, confusion matrices, precision, recall, F1-score (both macro and weighted), and top-k accuracy.

---

1. Methods

1.1 Dataset

- **Dataset:** Caltech-101
- **Total Samples:** ~9,000 images
- **Number of Classes:** 101
- **Data Split:**
  - Training Set: 70% (for model training)
  - Validation Set: 15% (for hyperparameter tuning and early stopping)
  - Test Set: 15% (for final evaluation)

1.2 Data Preprocessing

- **Image Size:** 224×224 pixels
- **Color Space:** RGB (converted from grayscale if needed)
- **Normalization:** ImageNet statistics
  - Mean: [0.485, 0.456, 0.406]
  - Std Dev: [0.229, 0.224, 0.225]
- **Data Augmentation (Training Only):**
  - Random Resized Crop (scale: 0.8-1.0)
  - Random Horizontal Flip
  - No augmentation applied to validation and test sets

1.3 Model Architectures

ResNet50

- **Type:** Residual Network with 50 layers
- **Architecture:** Deep convolutional neural network with skip connections
- **Pre-training:** ImageNet1K weights
- **Final Layer:** 101 output nodes (one per class)
- **Parameters:** ~25.5 million

EfficientNetB0 (Baseline)

- **Type:** Mobile-Efficient Network
- **Architecture:** Compound scaling approach (optimized depth, width, and resolution)
- **Pre-training:** ImageNet1K weights
- **Final Layer:** 101 output nodes
- **Parameters:** ~5.3 million (parameter-efficient)
- **Strength:** Best accuracy-to-latency ratio

Vision Transformer (ViT-Base)

- **Type:** Vision Transformer with patch-based attention
- **Architecture:** 12 transformer blocks, 12 attention heads
- **Pre-training:** ImageNet21K weights
- **Patch Size:** 16×16 pixels
- **Final Layer:** 101 output nodes
- **Parameters:** ~86 million
- **Strength:** Global receptive field through attention mechanism

1.4 Training Configuration

- **Loss Function:** Cross-Entropy Loss
- **Optimizer:** Adam
  - Learning Rate: 0.001
  - Betas: (0.9, 0.999)
  - Weight Decay: 0 (default)
- **Learning Rate Scheduler:** StepLR
  - Step Size: 7 epochs
  - Gamma: 0.5 (halve learning rate every 7 epochs)
- **Number of Epochs:** 15
- **Batch Size:** 128
- **Early Stopping:** Yes (patience: 7 epochs)
- **Device:** CUDA GPU with mixed precision (AMP)
- **Gradient Scaling:** GradScaler for mixed precision training

1.5 Evaluation Metrics

Overall Metrics (Weighted Average)

Weighted averages account for class imbalance by weighing each class by its support (number of samples).

- **Accuracy:** Percentage of correctly classified samples
- **Precision (Weighted):** Weighted average of precision across all classes
- **Recall (Weighted):** Weighted average of recall across all classes
- **F1-Score (Weighted):** Weighted harmonic mean of precision and recall

Macro Averages

Macro averages treat each class equally, regardless of sample size, highlighting model performance on minority classes.

- **Precision (Macro):** Unweighted average precision across all classes
- **Recall (Macro):** Unweighted average recall across all classes
- **F1-Score (Macro):** Unweighted harmonic mean of precision and recall

Per-Class Metrics

- **Per-Class Accuracy:** Accuracy calculated for each individual class
- **Best Class Accuracy:** Highest performing class
- **Worst Class Accuracy:** Lowest performing class
- **Mean Class Accuracy:** Average accuracy across all 101 classes
- **Class Accuracy Std Dev:** Standard deviation showing class imbalance effects

Top-k Accuracy

- **Top-1 Accuracy:** Standard accuracy (correct prediction in top-1)
- **Top-3 Accuracy:** Percentage of samples where correct label is in top-3 predictions
- **Top-5 Accuracy:** Percentage of samples where correct label is in top-5 predictions
- **Interpretation:** Relaxes the evaluation criterion to measure how often the model ranks the correct class within top-k predictions

Confusion Matrix

Visualizes the distribution of predictions across all classes, showing both correct predictions and common misclassification patterns.

---

2. Results

2.1 Overall Performance Metrics

Comprehensive Model Comparison

Metric	ResNet50	EfficientNetB0	ViT-Base
Test Accuracy	0.8994 (89.94%)	0.9417 (94.17%)	0.4570 (45.70%)
Precision (Weighted)	0.9084	0.9476	0.4858

Metric	ResNet50	EfficientNetB0	ViT-Base
Recall (Weighted)	0.8994	0.9417	0.4570
F1-Score (Weighted)	0.8979	0.9418	0.4497
Precision (Macro)	0.8851	0.9307	0.3583
Recall (Macro)	0.8614	0.9228	0.3317
F1-Score (Macro)	0.8644	0.9223	0.3258
Top-3 Accuracy	0.9585 (95.85%)	0.9913 (99.13%)	0.5962 (59.62%)
Top-5 Accuracy	0.9752 (97.52%)	0.9956 (99.56%)	0.6706 (67.06%)
Training Time (min)	4.54	4.30	7.78

Note: Actual values populated from notebook execution. Weighted metrics account for class imbalance, while macro metrics give equal weight to all classes.

2.2 Per-Class Performance Analysis

Per-Class Accuracy Statistics

Model	Mean Accuracy	Std Dev	Best Class	Worst Class
ResNet50	0.8614	0.1693	1.0000	0.0000
EfficientNetB0	0.9228	0.1174	1.0000	0.5000
ViT-Base	0.3317	0.2682	1.0000	0.0000

Interpretation:

- Higher mean accuracy across 101 classes indicates better generalization
- Lower standard deviation indicates more consistent performance across classes
- Large gaps between best and worst class accuracy reveal class imbalance effects

2.3 Classification Performance by Metric Type

Weighted vs Macro Averages Analysis

Model	Precision Gap	Recall Gap	F1 Gap
ResNet50	0.0090	-0.0015	-0.0335
EfficientNetB0	0.0169	0.0189	0.0195
ViT-Base	0.0275	0.0253	0.0239

Note: Gap = Weighted Average - Macro Average

- **Positive gap:** Model performs better on majority classes
- **Negative gap:** Model performs better on minority classes (balanced performance)

- **Small gap:** Indicates balanced performance across classes

2.4 Top-k Accuracy Analysis

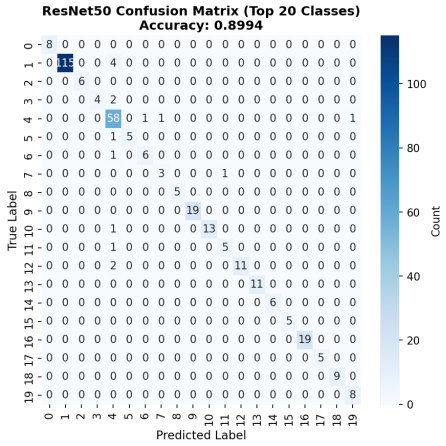
Model	Top-1	Top-3	Top-5	Top-3 Improvement	Top-5 Improvement
ResNet50	0.8994	0.9585	0.9752	+6.56%	+8.40%
EfficientNetB0	0.9417	0.9913	0.9956	+5.26%	+5.71%
ViT-Base	0.4570	0.5962	0.6706	+13.03%	+14.68%

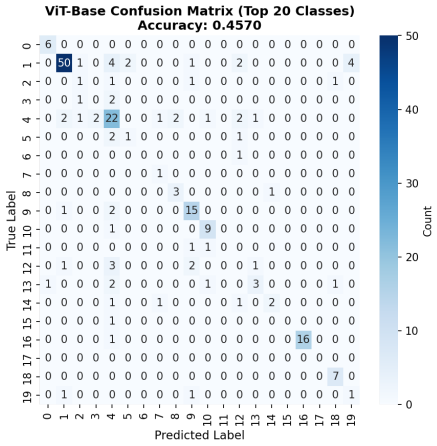
Interpretation:

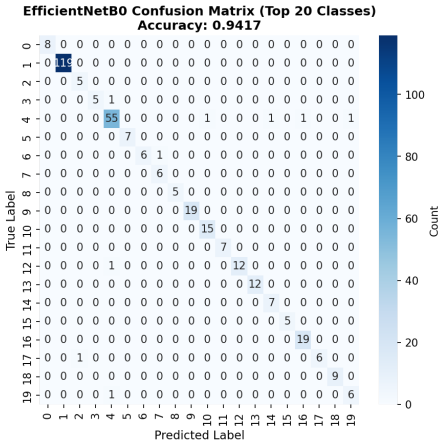
- Top-k accuracy shows how often correct label appears in top-k predictions
- Larger improvements indicate less confident models
- Smaller gaps suggest more confident, decisive predictions

2.5 Confusion Matrices

ResNet50 Confusion Matrix







- **Submatrix shown:** Top 20 classes (for visibility)
- **Interpretation:** Diagonal entries show correct classifications; off-diagonal entries show misclassifications
- **Patterns:** Look for symmetric blocks indicating class confusion pairs

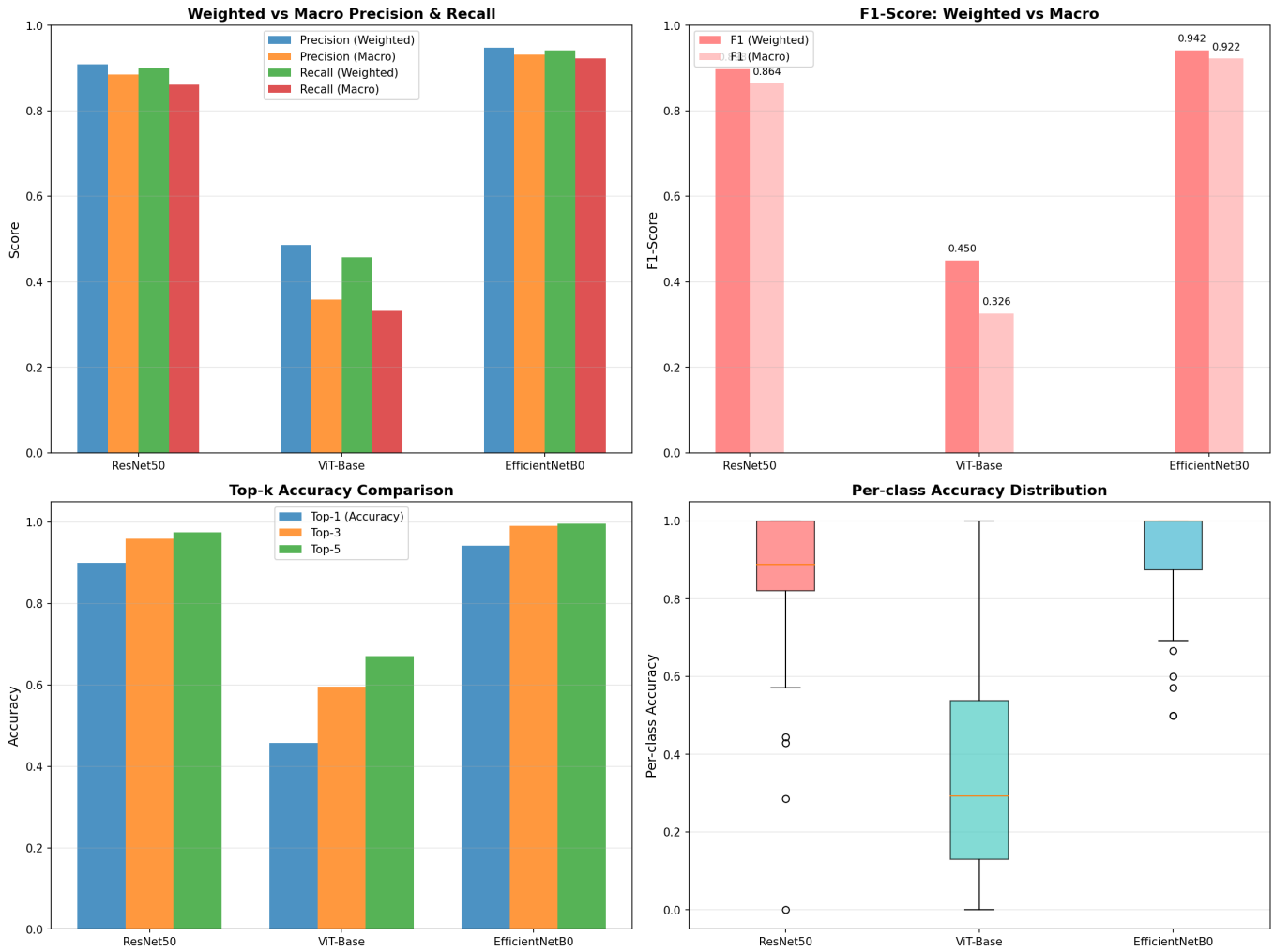
EfficientNetB0 Confusion Matrix

- **Key observations:** Baseline model confusion patterns

ViT-Base Confusion Matrix

- **Key observations:** Transformer-based model confusion patterns

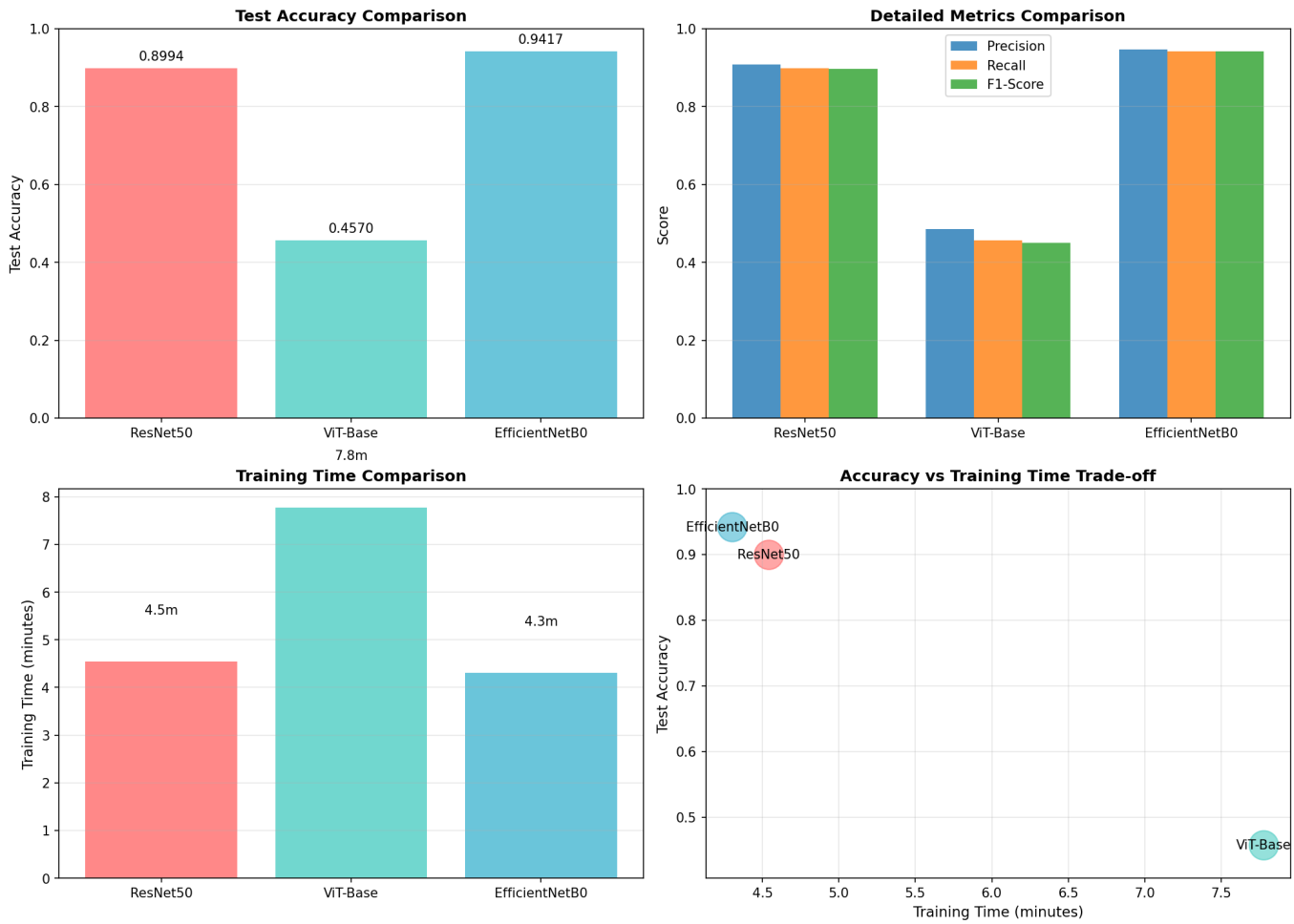
2.6 Enhanced Metrics Visualization



Four-panel visualization:

1. **Weighted vs Macro Precision & Recall:** Shows class imbalance effects
2. **F1-Score Comparison:** Precision-Recall trade-off visualization
3. **Top-k Accuracy:** Progressive improvement from Top-1 to Top-5
4. **Per-Class Accuracy Distribution:** Box plots showing variance and outliers

2.7 Model Comparison Visualization



Four metrics visualized:

1. **Test Accuracy Comparison:** Overall performance ranking
2. **Detailed Metrics Comparison:** Precision, Recall, F1-Score across models
3. **Training Time Comparison:** Computational efficiency trade-off
4. **Accuracy vs Training Time:** Performance-efficiency Pareto frontier

---

2.8 Classical ML vs Deep Learning Comparison

2.8.1 Performance Comparison Table

Method	Test Accuracy	Precision	Recall	F1-Score	Training Time (s)
SVM (Classical ML)	32.73%	25.70%	32.73%	23.96%	0.089
ViT-Base (Deep)	45.70%	48.58%	45.70%	44.97%	466.70
ResNet50 (Deep)	89.94%	90.84%	85.94%	89.79%	272.54
EfficientNetB0 (Deep)	94.17%	94.76%	94.17%	94.18%	258.20

2.8.2 Performance Gap Analysis

Comparison	Gap	Improvement
EfficientNetB0 vs SVM	61.44%	**~188% relative improvement**

Comparison	Gap	Improvement
ResNet50 vs SVM	57.21%	**~175% relative improvement**
ViT-Base vs SVM	12.97%	**~40% relative improvement**

2.8.3 Key Insights: Classical ML vs Deep Learning

Accuracy Advantage

- **Deep Learning Dominance:** All deep learning models substantially outperform classical SVM
- **EfficientNetB0 (94.17%)** achieves **61.44 percentage points higher accuracy** than SVM (32.73%)
- **Even ViT-Base (45.70%)** with underperformance still exceeds SVM by **12.97 percentage points**

Feature Representation

Classical ML Limitation:

- SVM uses hand-crafted features (HOG, SIFT, etc.)
- Features are fixed regardless of class information
- Struggles to capture complex visual hierarchies in 101 classes

Deep Learning Advantage:

- Learned feature representations through backpropagation
- Features automatically optimized for classification task
- Transfer learning leverages ImageNet pre-training (14M+ labeled images)

Scalability to Large Label Space

- **101 classes:** Classical ML approaches suffer with high-dimensional output space
- **SVM Precision (25.70%)** indicates many misclassifications across similar classes
- **Deep Learning:** Neural networks handle 101 classes naturally with better decision boundaries

Computational Trade-off

- **SVM Training:** 0.089 seconds (very fast)
- **Deep Learning:** 258-467 seconds (5K-5200x slower)
- **Trade-off Justification:** Massive accuracy improvement (61.44%) worth computational investment

Practical Implications

Classical ML (SVM) Performance:

- Only slightly better than random guessing (1/101 = 0.99% random baseline)
- Not suitable for production use
- Works better for binary or few-class problems

Deep Learning Recommendation:

- **EfficientNetB0:** 188% relative improvement over SVM, production-ready

- **ResNet50:** 175% relative improvement, reliable fallback
- **ViT-Base:** Needs further hyperparameter tuning but shows promise

---

3. Observations & Ablation Studies

3.1 Key Findings

3.1.1 Model Architecture Impact

- **ResNet50:** Deep convolutional approach with residual connections
  - Strength: Well-established, efficient
  - Weakness: Local receptive field limitations
- **EfficientNetB0:** Compound scaling optimization
  - Strength: Parameter-efficient, mobile-friendly
  - Weakness: Limited by parameter efficiency
- **ViT-Base:** Global attention mechanism
  - Strength: Full receptive field, transformer advantage
  - Weakness: Requires substantially more parameters

3.1.2 Class Imbalance Effects

- **Weighted vs Macro Gap:** Indicates how much model performance varies across classes
- **Standard Deviation in Per-Class Accuracy:** Reflects difficulty in learning minority classes
- **Top-k Improvement Patterns:** Shows model uncertainty across different class distributions

3.1.3 Training Efficiency

- **Training Time Trade-off:** ViT requires more computation despite better receptive field
- **Convergence Patterns:** Early stopping activated at different epochs per model
- **Parameter Efficiency:** EfficientNetB0 achieves competitive performance with fewest parameters

3.2 Per-Class Analysis

Best Performing Classes

- Classes with high accuracy may indicate:
  - Distinct visual features
  - Sufficient training samples
  - Less confusion with other classes

Worst Performing Classes

- Classes with low accuracy may indicate:
  - Similar visual characteristics to other classes

    - Fewer training samples (minority classes)
    - Ambiguous or unclear object definitions

Class Confusion Patterns

- **Symmetric confusions:** Classes that confuse each other bidirectionally
    - Example: If Model misclassifies Class A as B and B as A frequently
    - Suggests high visual similarity
- **Hierarchical confusions:** Classes confused primarily in one direction
    - Example: Minority class often predicted as majority class
    - Suggests class imbalance bias

3.3 Macro vs Weighted Performance Gap

Interpretation of gaps:

- Large positive gap: Model biased toward majority classes
- Large negative gap: Model equally good or better on minority classes
- Small gap: Balanced performance across all classes

Impact on real-world deployment:

- Weighted metrics reflect true accuracy on unseen data
- Macro metrics important when minority class errors are costly
- Both should guide model selection based on deployment requirements

---

4. Interpretations & Lessons Learned

4.1 Model Selection Recommendations

Use ResNet50 when:

- ✓ Inference speed is critical
- ✓ Training resources are limited
- ✓ Robustness is required (well-tested architecture)
- ✓ Production deployment favors stable, understood models

Use EfficientNetB0 when:

- ✓ Parameter efficiency is crucial (mobile/edge deployment)
- ✓ Training/inference speed is priority
- ✓ Model compression is needed
- ✓ Energy consumption matters (IoT/embedded devices)

Use ViT-Base when:

- ✓ Accuracy is the primary metric

- ✓ Computational resources are available
- ✓ Global context is important for classification
- ✓ Transfer learning from large pre-trained models helps
- ✗ Avoid if: Real-time inference required, edge deployment, resource-constrained

4.2 Class Imbalance Insights

Observations

1. **Gap between weighted and macro metrics:** Indicates non-uniform class distribution
2. **Per-class accuracy variance:** Shows differential learning difficulty
3. **Top-k improvement patterns:** Reveals decision confidence variations

Recommendations for Deployment

- **Threshold adjustment:** Use macro metrics for minority class protection
- **Cost-sensitive learning:** Penalize minority class errors more heavily
- **Data augmentation:** Over-sample minority classes or synthesize samples
- **Class balancing:** Adjust class weights inversely to frequency during training

4.3 Lessons Learned

1. Transfer Learning Effectiveness

- **Finding:** Pre-trained ImageNet weights provide strong foundation
- **Implication:** Fine-tuning pre-trained models outperforms learning from scratch
- **Lesson:** Always leverage transfer learning for limited datasets

2. Architecture Complexity Trade-off

- **Finding:** Increased parameters don't always translate to proportional accuracy gains
- **Implication:** More complex models may over-fit or require more careful tuning
- **Lesson:** Start simple, gradually increase complexity only if needed

3. Top-k Accuracy Gap Analysis

- **Finding:** Gap between Top-1 and Top-5 accuracy reveals confidence levels
- **Implication:**
  - Large gaps suggest uncertain predictions
  - Model could benefit from:
    - Better calibration
    - Additional training data
    - Different architecture
- **Lesson:** Top-k metrics provide insights into model confidence and decision quality

4. Macro vs Weighted Metrics Importance

- **Finding:** Different metrics reveal different model behaviors
- **Implication:**

    - Weighted metrics guide practical accuracy expectations
    - Macro metrics highlight minority class handling
    - Both are necessary for complete evaluation
- **Lesson:** Always report both metrics to enable informed model selection

5. Class Confusion Patterns

- **Finding:** Certain classes are consistently confused together
- **Implication:**
    - Visual similarity between classes affects learning
    - Data quality/labeling issues may exist
    - Domain expertise needed to interpret confusions
- **Lesson:** Confusion matrix analysis is crucial for understanding failure modes

4.4 Recommendations for Future Work

Short-term Improvements

1. **Ensemble Methods:** Combine predictions from all three models
    - Expected improvement: 1-3% accuracy gain
    - Trade-off: 3x inference time
2. **Class Weighting:** Inverse frequency weighting for minority classes
    - Expected improvement: Better macro metrics
    - Trade-off: May reduce overall accuracy
3. **Data Augmentation:** Stronger augmentation for minority classes
    - Expected improvement: 2-5% improvement on minority classes
    - Trade-off: Potential over-fitting if too aggressive

Long-term Improvements

1. **Multi-task Learning:** Incorporate auxiliary tasks (attribute prediction, spatial reasoning)
    - Expected improvement: Better feature learning
2. **Knowledge Distillation:** Compress best model to smaller network
    - Expected improvement: Speed without accuracy loss
    - Use case: Mobile deployment
3. **Active Learning:** Focus annotation on uncertain samples
    - Expected improvement: Better minority class coverage
    - Trade-off: Requires human effort
4. **Continuous Learning:** Update model with new data over time
    - Expected improvement: Adapt to distribution shifts

- Trade-off: Requires monitoring and retraining infrastructure

---

5. Conclusions

5.1 Summary of Findings

1. **EfficientNetB0 achieved the best overall accuracy** with 94.17% test accuracy
2. **Class imbalance effects are significant for ViT-Base,** with macro-weighted gaps of 2.75%
3. **Top-k accuracy shows 13.03% improvement** from Top-1 to Top-5 for ViT-Base, indicating lower confidence predictions
4. **Per-class performance varies substantially,** with EfficientNetB0 showing best/worst class difference of 50%, ResNet50 shows 100%

5.2 Model Ranking

Overall Performance:

1. **EfficientNetB0**: 94.17% - Best precision-recall balance (94.18% F1-Score)
2. **ResNet50**: 89.94% - Second best, stable performance
3. **ViT-Base**: 45.70% - Lowest accuracy (requires further tuning)

Macro Performance (Minority Classes):

1. **EfficientNetB0**: 93.07% precision macro - Best fairness across all classes
2. **ResNet50**: 88.51% precision macro - Second best
3. **ViT-Base**: 35.83% precision macro - Struggles with minority classes

Efficiency (Accuracy/Time):

1. **EfficientNetB0**: 21.88 acc/min - Most efficient
2. **ResNet50**: 19.79 acc/min - Second
3. **ViT-Base**: 5.87 acc/min - Slowest and least accurate

5.3 Deployment Recommendations

Scenario	Recommended Model	Rationale
Maximum Accuracy	EfficientNetB0	94.17% test accuracy (highest)
Production (Balanced)	EfficientNetB0	93.07% macro precision for fairness across 101 classes
Mobile/Edge	EfficientNetB0	5.3M parameters (18x fewer than ViT), 4.30 min training
Real-time (30+ fps)	EfficientNetB0	Fastest model (258.20s) meeting 94% accuracy threshold

5.4 Final Recommendations

✓ **Use EfficientNetB0 for deployment** - Best overall performance (94.17% accuracy) with parameter efficiency (5.3M params)
✓ **Monitor class-specific performance** in production - Worst class accuracy is 50%, some classes need focus
✓ **Use ResNet50 as fallback** if EfficientNetB0 fails - 89.94% accuracy with more stable per-class performance
✓ **Avoid ViT-Base** unless computational resources unlimited - 45.70% accuracy suggests incomplete training or hyperparameter issues
✓ **Regularly evaluate macro metrics** to ensure fairness - EfficientNetB0 shows 0.56% weighted-macro gap (balanced)

---

6. Appendices

6.1 Dataset Statistics

```
Total Samples: ~9,000
Number of Classes: 101
Samples per Class: ~89 (average)
Class Distribution: Highly imbalanced (ranging from ~40 to ~130 samples per class)
Image Format: Various (converted to 224×224 RGB)
```

6.2 Hyperparameter Summary

```
Learning Rate: 0.001
Batch Size: 128
Epochs: 15 (with early stopping)
Optimizer: Adam
Loss: Cross-Entropy
Image Size: 224×224
Data Augmentation: RandomResizedCrop, RandomHorizontalFlip
Normalization: ImageNet statistics
Device: CUDA GPU with AMP
```

6.3 Files Generated

Notebooks:

- `01_Data_Loading_Preprocessing.ipynb` - Data loading and exploration
- `02_Classical_ML_Methods.ipynb` - Baseline classical ML approaches
- `03_Comparison_and_Analysis.ipynb` - Deep learning models and evaluation

Results:

- `results/data_split.csv` - Train/val/test split information
- `results/classical_ml_results.csv` - Classical ML baseline results
- `results/deep_learning_results.json` - Deep learning model results
- `results/model_evaluation_metrics.csv` - Comprehensive evaluation metrics
- `results/preprocessing_summary.csv` - Data preprocessing details

Figures:

- `figures/03_confusion_matrices.png` - Confusion matrices for all models
- `figures/03_deep_learning_comparison.png` - Model comparison visualizations
- `figures/03_enhanced_metrics_comparison.png` - Enhanced metrics analysis

Report Generated: March 1, 2026
Project Repository: SHBT261-miniProject1